

Message Authentication Codes

CS 171: Cryptography

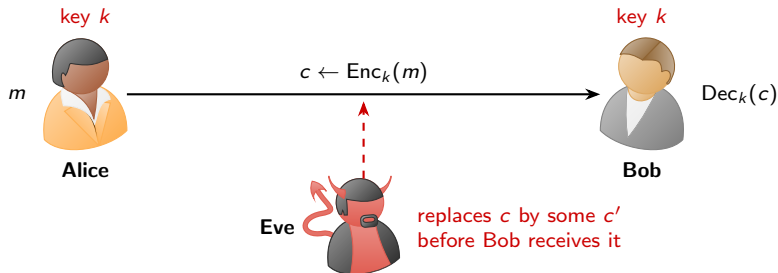
Sanjam Garg

September 29, 2026

Overview

- 1 Integrity vs. Secrecy
- 2 Defining MACs
- 3 Constructing a Fixed-Length MAC
- 4 From Fixed-Length to Arbitrary-Length
- 5 CBC-MAC
- 6 Summary

Integrity vs. Secrecy



Encryption does not give integrity

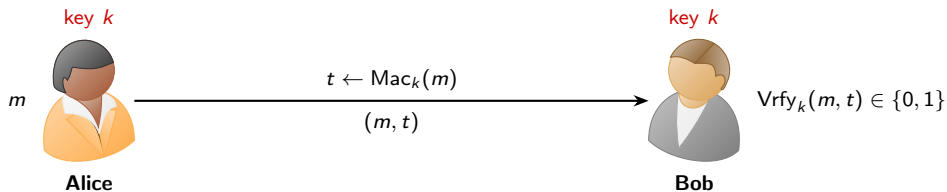
Take the CPA-secure $\text{Enc}_k(m) = \langle r, F_k(r) \oplus m \rangle$ from lecture 5. Flipping bit i of the second part flips bit i of the message Bob decrypts — and Bob cannot notice.

Two distinct goals

Secrecy: Eve learns nothing about m .

Integrity: Eve cannot make Bob accept a message Alice never sent.

Message Authentication Code (MAC)



Idea

Alice attaches a short **tag** t to the message; Bob checks it with the shared key k . Without k , an adversary should not be able to produce a valid tag on a message Alice never tagged — so tampering is detected.

MACs give integrity only: m is sent in the clear.

MACs — Formally

Definition 1 (Message Authentication Code)

A MAC is a triple $\Pi = (\text{Gen}, \text{Mac}, \text{Vrfy})$ of PPT algorithms:

- $\text{Gen}(1^n)$: outputs a key k .
- $\text{Mac}_k(m)$: outputs a tag t (possibly randomized).
- $\text{Vrfy}_k(m, t)$: outputs 0 or 1 (deterministic).

Correctness

For all n , all $k \leftarrow \text{Gen}(1^n)$ and all $m \in \{0, 1\}^*$: $\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1$.

Canonical verification

If Mac is *deterministic*, $\text{Vrfy}_k(m, t)$ can simply recompute: output 1 iff $\text{Mac}_k(m) = t$.

Unforgeability

Game $\text{MacForge}_{\mathcal{A}, \Pi}(n)$

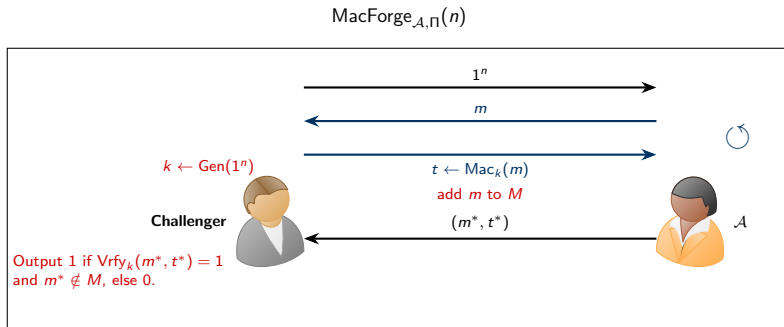
- 1 $k \leftarrow \text{Gen}(1^n)$. $\mathcal{A}$ is given 1^n and **oracle access** to $\text{Mac}_k(\cdot)$. Let M be the set of messages $\mathcal{A}$ queries.
- 2 $\mathcal{A}^{\text{Mac}_k(\cdot)}$ outputs a pair (m^*, t^*) .
- 3 Output 1 if $\text{Vrfy}_k(m^*, t^*) = 1$ and $m^* \notin M$, and 0 otherwise.

Definition 2 (EU-CMA)

$\Pi = (\text{Gen}, \text{Mac}, \text{Vrfy})$ is **existentially unforgeable under an adaptive chosen-message attack** (*eu-cma-secure*) if for all PPT $\mathcal{A}$:

$$\Pr [\text{MacForge}_{\mathcal{A}, \Pi}(n) = 1] \leq \text{negl}(n).$$

MacForge _{$\mathcal{A}, \Pi$} : Challenger vs. Adversary

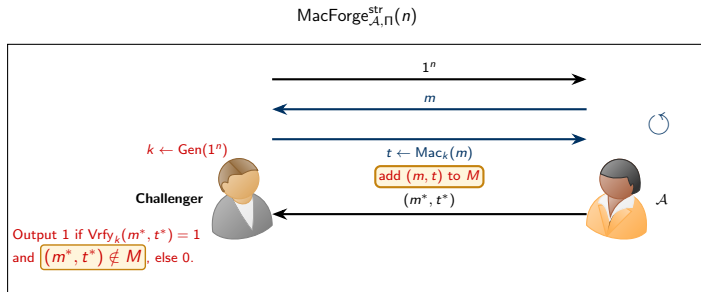


Remark

Existential: $\mathcal{A}$ wins with a valid tag on *any* new message, even a meaningless one.

Adaptive chosen-message: $\mathcal{A}$ picks each query after seeing the previous tags.

Strong Unforgeability



Strong unforgeability (suf-cma)

Π is **strongly unforgeable** if for all PPT $\mathcal{A}$: $\Pr[\text{MacForge}_{\mathcal{A}, \Pi}^{\text{str}}(n) = 1] \leq \text{negl}(n)$.
 $\mathcal{A}$ now also wins with a *new tag* on an *already-queried* message.

When is there no difference? For a deterministic Mac with canonical verification, each m has exactly one valid tag, so eu-cma and suf-cma coincide.

MAC Construction (for Fixed-Length Messages)

Construction from a PRF

Let $F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a PRF. For messages $m \in \{0, 1\}^n$:

- $\text{Gen}(1^n)$: sample $k \leftarrow \{0, 1\}^n$.
- $\text{Mac}_k(m)$: output the tag $t := F_k(m)$.
- $\text{Vrfy}_k(m, t)$: canonical — output 1 iff $F_k(m) = t$.

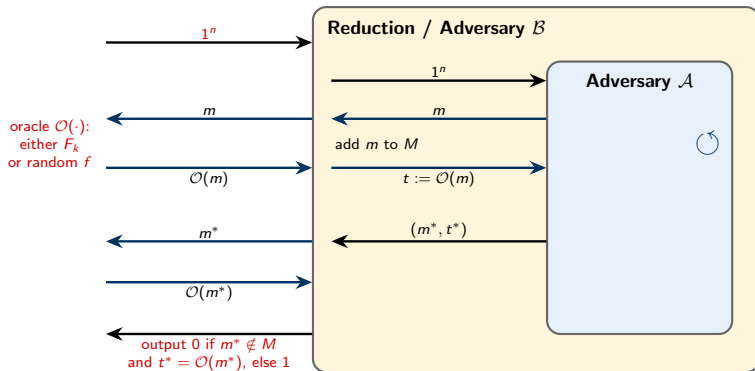
Theorem 3

If F is a PRF, then this is an eu-cma-secure MAC for messages in $\{0, 1\}^n$.

Proof by reduction

Given a PPT adversary $\mathcal{A}$ that forges tags, construct a PPT adversary $\mathcal{B}$ “breaking” the PRF F .

Proof by Reduction

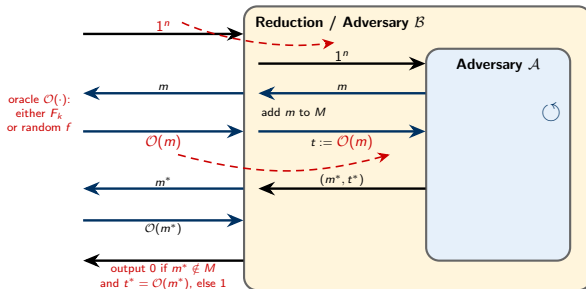


Given / To prove

Given: $\Pr[\text{MacForge}_{\mathcal{A}, \Pi}(n) = 1] \geq \epsilon(n)$ for non-negligible ϵ .

To prove: $\Pr[\text{PRF}_{\mathcal{B}, F}(n) = 1] \geq \frac{1}{2} + \delta(n)$ for non-negligible δ .

Proof of Security: The Reduction $\mathcal{B}$



$\mathcal{B}$, given 1^n and oracle access to $\mathcal{O}(\cdot)$

- 1 Run $\mathcal{A}(1^n)$; start with $M := \emptyset$.
- 2 On a query m from $\mathcal{A}$: query $\mathcal{O}(m)$, return $t := \mathcal{O}(m)$, and add m to M .
- 3 When $\mathcal{A}$ outputs (m^*, t^*) : query $\mathcal{O}(m^*)$.
- 4 Output 0 (" $\mathcal{O}$ is F_k ") if $m^* \notin M$ and $t^* = \mathcal{O}(m^*)$; else 1 (" $\mathcal{O}$ is random").

Proof of Security: Two Cases

Case $\mathcal{O} = F_k$ (pseudorandom)

$\mathcal{B}$ outputs 0 exactly when $\mathcal{A}$ forges:
 $\Pr[\mathcal{B} \text{ correct}] \geq \epsilon(n)$.

Every tag $\mathcal{B}$ returns is $F_k(m)$ — exactly $\text{Mac}_k(m)$. So the view of $\mathcal{A}$ is exactly as in $\text{MacForge}_{\mathcal{A}, \Pi}(n)$, and $\mathcal{B}$'s check is exactly its win condition.

Case $\mathcal{O} = f$ (truly random)

$\mathcal{A}$ forges only by guessing $f(m^*)$:
 $\Pr[\mathcal{B} \text{ correct}] \geq 1 - \frac{1}{2^n}$.

If $m^* \notin M$, then $f(m^*)$ was never queried: it is uniform in $\{0, 1\}^n$ and independent of $\mathcal{A}$'s view, so $\Pr[t^* = f(m^*)] = 2^{-n}$. $\mathcal{B}$ is wrong (outputs 0) only in this event.

Proof of Security: Putting It Together

Success probability of $\mathcal{B}$

$$\begin{aligned}\Pr[\text{PRF}_{\mathcal{B},F}(n) = 1] &= \frac{1}{2} \Pr[\mathcal{B} \text{ correct} \mid \mathcal{O} = F_k] + \frac{1}{2} \Pr[\mathcal{B} \text{ correct} \mid \mathcal{O} = f] \\ &\geq \frac{1}{2} \epsilon(n) + \frac{1}{2} \left(1 - \frac{1}{2^n}\right) = \frac{1}{2} + \frac{\epsilon(n)}{2} - \frac{1}{2^{n+1}}.\end{aligned}$$

If $\epsilon(n)$ is non-negligible then so is $\epsilon(n)/2 - 1/2^{n+1}$, contradicting that F is a PRF. $\square$

Remark (running time)

$\mathcal{B}$ runs $\mathcal{A}$ once, makes one oracle query per query of $\mathcal{A}$ plus one more, and keeps the list M — so $\mathcal{B}$ is PPT given $\mathcal{A}$ is PPT.

Arbitrary-Length MACs: Attempt 1

Construct Mac' (arbitrary length) from Mac (for messages in $\{0, 1\}^n$)

$\text{Mac}'_k(m)$:

- Parse m as $m_1 \cdots m_d$, where each m_i has length n .
- Output $t_1 \cdots t_d$, where $t_i = \text{Mac}_k(m_i)$ for each i .

Secure?

No! Reorder the blocks

Query $m_1 \| m_2$ with $m_1 \neq m_2$, get $t_1 \| t_2$. Output the forgery $(m_2 \| m_1, t_2 \| t_1)$: a valid tag on a message never queried.

Arbitrary-Length MACs: Attempt 2

Add a block index

$\text{Mac}'_k(m)$:

- Parse m as $m_1 \cdots m_d$, where each m_i has length $n/2$.
- Output $t_1 \cdots t_d$, where $t_i = \text{Mac}_k(i \parallel m_i)$ for each i (i written with $n/2$ bits).

Secure?

No! Mix and match blocks from different messages

Query $m_1 \parallel m_2$ and $m'_1 \parallel m'_2$ (with $m_1 \neq m'_1$, $m_2 \neq m'_2$), get $t_1 \parallel t_2$ and $t'_1 \parallel t'_2$. Output $(m_1 \parallel m'_2, t_1 \parallel t'_2)$: every block carries the right index.

Arbitrary-Length MACs: Attempt 3

Add a per-message random value

$\text{Mac}'_k(m)$:

- Parse m as $m_1 \cdots m_d$, where each m_i has length $n/3$.
- Sample $r \leftarrow \{0, 1\}^{n/3}$.
- Output $(r, t_1 \cdots t_d)$, where $t_i = \text{Mac}_k(r \parallel i \parallel m_i)$ for each i .

Secure?

No! Drop blocks from the end

Query $m_1 \parallel m_2$, get $(r, t_1 \parallel t_2)$. Output $(m_1, (r, t_1))$: $t_1 = \text{Mac}_k(r \parallel 1 \parallel m_1)$ is exactly the tag Mac' would give m_1 with this r . The **length** is not authenticated.

Arbitrary-Length MACs: A Secure Construction

Construct Mac' (arbitrary length) from Mac (for messages in $\{0, 1\}^n$)

$\text{Mac}'_k(m)$:

- Parse m as $m_1 \cdots m_\ell$, where each m_i has length $n/4$.
- Sample $r \leftarrow \{0, 1\}^{n/4}$.
- Output $(r, t_1 \cdots t_\ell)$, where $t_i = \text{Mac}_k(r \parallel \ell \parallel i \parallel m_i)$ for each i .

(ℓ and i are written with $n/4$ bits each, so messages have fewer than $2^{n/4}$ blocks.)

- Each block tag binds the random r , the length ℓ , the index i , and the block m_i — one ingredient against each attack.
- Mac' is **randomized**! How do we define Vrfy' ?
 $\text{Vrfy}'_k(m, (r, t_1 \cdots t_\ell))$: parse m into ℓ blocks and output 1 iff $\text{Vrfy}_k(r \parallel \ell \parallel i \parallel m_i, t_i) = 1$ for every i .

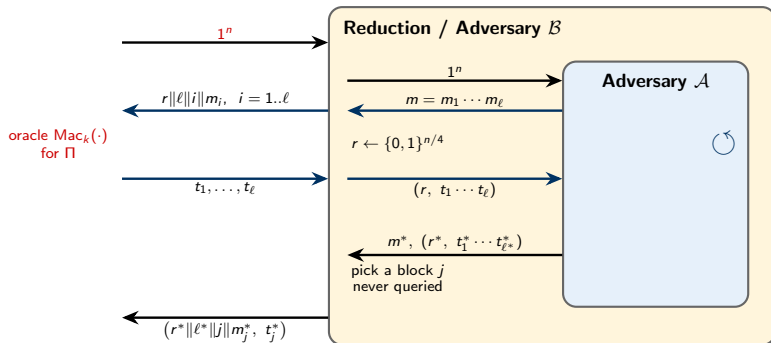
Theorem 4

If $\Pi = (\text{Gen}, \text{Mac}, \text{Vrfy})$ is eu-cma-secure for messages in $\{0, 1\}^n$, then $\Pi' = (\text{Gen}, \text{Mac}', \text{Vrfy}')$ is eu-cma-secure for messages of fewer than $2^{n/4}$ blocks.

Proof by reduction

Given a PPT adversary $\mathcal{A}$ that forges tags for Π' , construct a PPT adversary $\mathcal{B}$ that forges tags for Π .

Proof by Reduction

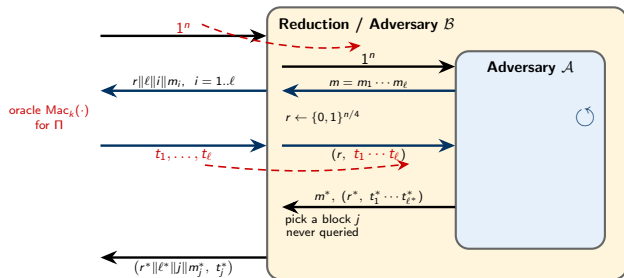


Given / To prove

Given: $\Pr[\text{MacForge}_{\mathcal{A}, \Pi'}(n) = 1] \geq \epsilon(n)$ for non-negligible ϵ .

To prove: $\Pr[\text{MacForge}_{\mathcal{B}, \Pi}(n) = 1] \geq \delta(n)$ for non-negligible δ .

Proof of Security: The Reduction $\mathcal{B}$



$\mathcal{B}$, given 1^n and oracle access to $\text{Mac}_k(\cdot)$

- ① Run $\mathcal{A}(1^n)$.
- ② On a query $m = m_1 \dots m_\ell$: sample $r \leftarrow \{0, 1\}^{n/4}$, query $t_i := \text{Mac}_k(r \parallel \ell \parallel i \parallel m_i)$ for each i , and return $(r, t_1 \dots t_\ell)$.
- ③ When $\mathcal{A}$ outputs $(m^*, (r^*, t_1^* \dots t_{\ell^*}^*))$: find a j such that $r^* \parallel \ell^* \parallel j \parallel m_j^*$ was never queried, and output $(r^* \parallel \ell^* \parallel j \parallel m_j^*, t_j^*)$.

Proof of Security: Which Block Is New?

Suppose $\mathcal{A}$ wins, and the $r_1, r_2, \dots$ that $\mathcal{B}$ sampled for the queries $m_1, m_2, \dots$ are all **distinct**.

- **Case I:** $r^* \neq r_i$ for all i .

No queried block starts with r^* , so $j = 1$ works.

- **Case II:** $r^* = r_i$ (for exactly one i), but $\ell^* \neq \ell_i$.

Every queried block starting with r^* came from query i and contains $\ell_i \neq \ell^*$, so $j = 1$ works.

- **Case III:** $r^* = r_i$ and $\ell^* = \ell_i$, but $m^* \neq m_i$.

Then $m_j^* \neq m_{i,j}$ for some j ; the only queried block starting with $r^* \parallel \ell^* \parallel j$ ends in $m_{i,j}$, so this j works.

In every case

Vrfy' accepted, so $\text{Vrfy}_k(r^* \parallel \ell^* \parallel j \parallel m_j^*, t_j^*) = 1$ on a block $\mathcal{B}$ never queried: $\mathcal{B}$ wins $\text{MacForge}_{\mathcal{B}, \Pi}(n)$.

Proof of Security: Putting It Together

Success probability of $\mathcal{B}$

$\mathcal{B}$'s answers are computed exactly as Mac'_k , so $\mathcal{A}$'s view is as in $\text{MacForge}_{\mathcal{A}, \Pi'}(n)$. Let $q(n)$ bound the number of queries and Distinct be the event that all r_i are distinct:

$$\Pr[\neg \text{Distinct}] \leq \binom{q(n)}{2} \cdot 2^{-n/4} \leq q(n)^2 / 2^{n/4}.$$

$$\begin{aligned} \Pr[\text{MacForge}_{\mathcal{B}, \Pi}(n) = 1] &\geq \Pr[\text{MacForge}_{\mathcal{A}, \Pi'}(n) = 1 \wedge \text{Distinct}] \\ &\geq \Pr[\text{MacForge}_{\mathcal{A}, \Pi'}(n) = 1] - \Pr[\neg \text{Distinct}] \\ &\geq \epsilon(n) - \frac{q(n)^2}{2^{n/4}}. \end{aligned}$$

(The second step uses $\Pr[X] \leq \Pr[X \wedge \text{Distinct}] + \Pr[\neg \text{Distinct}]$.) If $\epsilon(n)$ is non-negligible then so is $\epsilon(n) - q(n)^2 / 2^{n/4}$ (since q is polynomial), contradicting that Π is eu-cma-secure. $\square$

Remark (running time)

$\mathcal{B}$ runs $\mathcal{A}$ once, makes ℓ oracle queries per query of $\mathcal{A}$, and searches its list of queried blocks once — so $\mathcal{B}$ is PPT given $\mathcal{A}$ is PPT.

Question

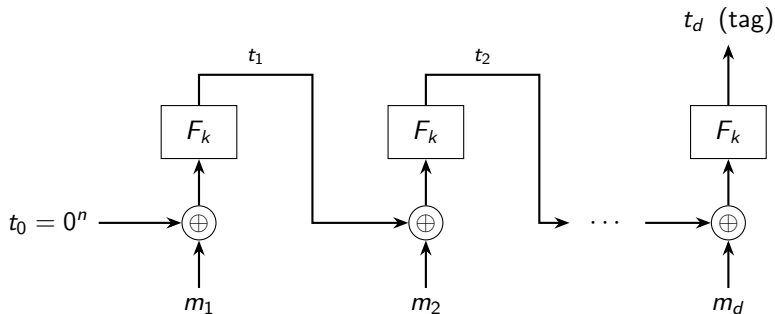
With $\text{Mac}_k = F_k$ from before (so each t_i has n bits), how long is the tag on a message of $\ell \cdot n$ bits?

Answer

The message has 4ℓ blocks of $n/4$ bits, each with its own n -bit tag: $4 \cdot \ell \cdot n$ bits (plus r) — **four times longer than the message itself!**

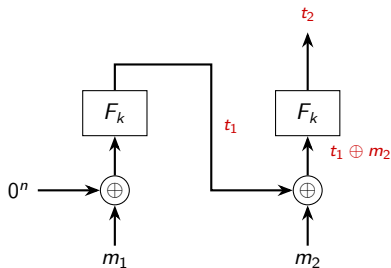
Can we get a short tag, independent of the message length?

CBC-MAC (Using a Block Cipher)



- Chain the blocks: $t_i = F_k(t_{i-1} \oplus m_i)$, with $t_0 = 0^n$; output only the final t_d .
- **The tag is only n bits**, whatever the length of m .
- **Secure for messages of different lengths? No!** An adversary can forge tags on longer messages.

Attack on CBC-MAC (Variable Length)



Length-extension forgery

- 1 Query the one-block message m_1 ; get $t_1 = F_k(m_1)$.
- 2 Query the one-block message $t_1 \oplus m_2$; get $t_2 = F_k(t_1 \oplus m_2)$.
- 3 Output $(m_1 \| m_2, t_2)$.

CBC-MAC on $m_1 \| m_2$ computes $F_k(F_k(m_1) \oplus m_2) = F_k(t_1 \oplus m_2) = t_2$, and $m_1 \| m_2$ was never queried.

CBC-MAC: Security for Fixed Length

Theorem 5

Let $\ell(\cdot)$ be a polynomial. If F is a PRF, then CBC-MAC is eu-cma-secure for messages of (fixed) length $\ell(n) \cdot n$.

Proof idea

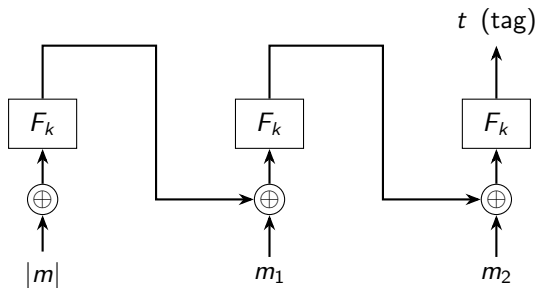
It suffices to show that $\text{CBC}_k(x_1, \dots, x_\ell) = F_k(F_k(\dots F_k(F_k(x_1) \oplus x_2) \dots) \oplus x_\ell)$ is itself a PRF on $\ell \cdot n$ -bit inputs; then the proof for $\text{Mac}_k(m) = F_k(m)$ applies unchanged.

- More is true: CBC_k is a PRF as long as its inputs come from a **prefix-free** set $P \subset (\{0, 1\}^n)^*$:
 - P does not contain the empty string;
 - no $x \in P$ is a prefix of another $x' \in P$.
- The attack used m_1 and $m_1 \| m_2$ — one a prefix of the other.
- We will not prove this here.

Handling Arbitrary-Length Messages (Method 1)

Method 1: prepend the length

For messages whose length is a multiple of n : the tag on m is the CBC-MAC of $|m| \parallel m$ (the length block followed by $m_1, m_2, \dots$).



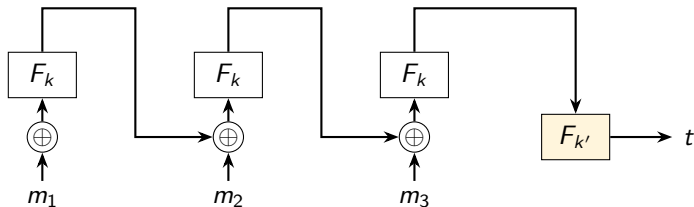
- **Why is this prefix-free?** Messages of different lengths already differ in the first block; messages of the same length cannot be proper prefixes of each other.

Handling Arbitrary-Length Messages (Method 2)

Method 2: MAC the CBC-MAC

For messages whose length is a multiple of n : compute CBC-MAC under key k , then apply F once more under an **independent** key k' :

$$t = F_{k'}(\text{CBC}_k(m_1, \dots, m_d)).$$



- Secure for messages of any length (multiple of n), and the length need not be known in advance — the message can be processed as it streams in.

Summary

- Encryption gives **secrecy**, not **integrity**; MACs detect tampering.
- A MAC is $(\text{Gen}, \text{Mac}, \text{Vrfy})$ with correctness; security is **eu-cma** via the game $\text{MacForge}_{\mathcal{A}, \Pi}(n)$ (and **suf-cma**, where a new tag on an old message also counts).
- **Fixed length**: $\text{Mac}_k(m) = F_k(m)$ for a PRF F ; a forger yields a PRF distinguisher, since against a random f it succeeds with probability only 2^{-n} .
- **Arbitrary length**: block-by-block tags fail (reorder / mix and match / drop blocks); tagging each $r \parallel \ell \parallel i \parallel m_i$ works, but the tag is $4\ell n$ bits.
- **CBC-MAC**: n -bit tags by chaining; secure for fixed length, but the basic version is insecure for variable lengths (length extension).
- Fixes: prepend the length (Method 1), or apply $F_{k'}$ to the CBC-MAC under a second key (Method 2).